Project Report
On
CI/CD Pipeline for Vue.js Application

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

S. No.	Student Name	Enrollment Number
1.	Aayush Pal	EN22CS301022
2.	Aayushee Dhote	EN22CS301024
3.	Aditi Shrestha	EN22CS301044
4.	Aditya Chouhan	EN22CS301049
5.	Aditya Singh Chouhan	EN22CS301066

Group Name: Group 09D1

Project Number: DO-09

Industry Mentor Name:

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 2025 - 26

CONTENTS

1. Problem Statement & Objectives
   1.1 Problem Statement
   1.2 Project Objectives
   1.3 Scope of the Project
2. Proposed Solution
   2.1 Key Features
   2.2 Overall Architecture and Workflow
   2.3 Tools & Technologies Used
3. Result & Outputs
   3.1 System Interface and User Experience
   3.2 Key Outcomes
4. Conclusion
5. Future Scope and Enhancements

1. Problem Statement & Objectives

1.1 Problem Statement

In modern software development, delivering applications quickly, reliably, and consistently is a major challenge. Traditional deployment approaches involve manual testing, building, and deployment processes, which are not only time-consuming but also highly error-prone. These limitations make it difficult for development teams to maintain quality and meet the fast-paced demands of enterprise environments.

In the case of a Vue.js-based Technology application, the absence of an automated pipeline leads to several issues such as inconsistent builds, deployment failures, lack of testing validation, and poor visibility into the development lifecycle. Manual deployments increase the risk of human errors and reduce system reliability.

To address these challenges, this project focuses on designing and implementing a **complete CI/CD pipeline** that automates the entire software delivery lifecycle — from code commit to production deployment — using modern DevOps tools such as Git, GitHub Actions, Docker, and Kubernetes on AWS EC2.

1.2 Project Objectives

The primary objectives of this project are:

- To automate the execution of unit tests on every code commit
- To implement automated Docker image creation and push to Docker Hub upon successful merge to the main branch.
- To deploy the application on a Kubernetes cluster running on AWS EC2, ensuring scalability and high availability.
- To design a structured CI/CD workflow using GitHub Actions with clearly separated stages for testing, building, and deployment.
- To ensure consistent runtime environments using containerization.
- To demonstrate real-world DevOps practices applicable in enterprise systems.

1.3 Scope of the Project

The scope of this project covers the complete CI/CD pipeline for a Vue.js-based application. It includes:

- Source code management using Git and GitHub for version control and collaboration
- Continuous Integration using GitHub Actions with automated testing on every code push
- Continuous Deployment triggered on merging code to the main branch
- Containerization of the application using Docker for consistency and portability
- Deployment of the application using Kubernetes on AWS EC2 infrastructure
- Configuration of Kubernetes Deployment and NodePort Service for external access

2. Proposed Solution

The proposed solution is an automated CI/CD pipeline for a Vue.js application. It uses DevOps tools to ensure fast, reliable, and consistent deployment. The system combines version control, testing, containerization, and Kubernetes deployment into a single workflow.

2.1 Key Features

The proposed CI/CD pipeline offers the following key features:

- Automated testing on every code push
- Automated Docker image creation and versioning
- Continuous deployment to Kubernetes cluster
- Self-healing deployment using Kubernetes
- Stateless and scalable frontend architecture
- Separation of CI (testing) and CD (deployment) stages
- Consistent environment across development and production

2.2 Overall Architecture and Workflow

2.2.1 High-Level System Architecture

The system is designed as a four-stage pipeline that covers the complete software delivery process, from development to deployment on the cloud. Each stage is clearly defined, enabling automation, easy tracking, and better error handling.

The four main stages of the architecture are described below:

- Development Stage — Developers write Vue.js application code on their local machines and push changes to a remote GitHub repository. This serves as the single source of truth for all application code.

- CI/CD Pipeline Stage — GitHub Actions automatically detects push events and triggers predefined workflow configurations. These workflows orchestrate dependency installation, automated unit testing using Vitest, and the production build process.
- Containerization Stage — Once a build is verified, Docker packages the application into a portable container image and pushes it to Docker Hub, a centralized container registry accessible across environments.
- Deployment Stage — Kubernetes retrieves the latest container image and deploys it on AWS EC2 instances. It also manages pod replication, rolling updates, and service availability to ensure continuous uptime.

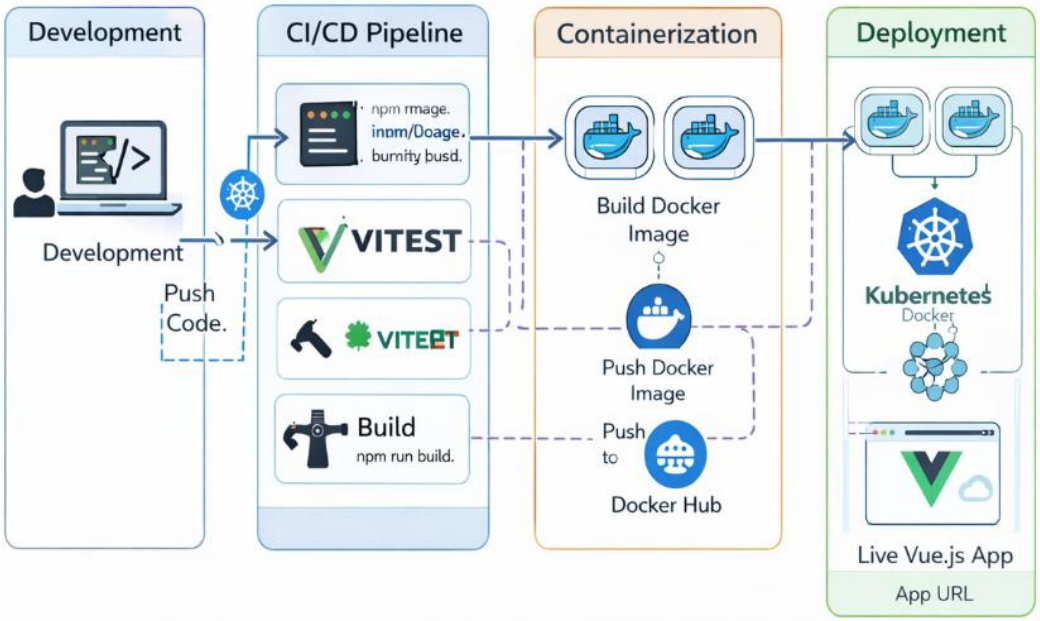


Figure 2.2.1: High-Level System Architecture

2.2.2 Step-by-Step Workflow

The end-to-end workflow follows a deterministic, event-driven sequence. Each step is triggered by the successful completion of the previous one, ensuring that no untested or unverified code reaches production. The complete workflow unfolds as follows:

1. Developer pushes code to a GitHub feature branch.
2. The CI pipeline is triggered automatically by the push event via GitHub Actions.
3. Dependencies are installed on the runner environment using npm install.

4. Unit tests are executed using the Vitest testing framework.
5. The application is compiled into production-ready static assets using npm run build.
6. A Docker image is created from the build output.
7. The Docker image is tagged with a version identifier and pushed to Docker Hub.
8. Kubernetes pulls the latest image from the registry.
9. The application is deployed as a set of managed pods across EC2 instances.
10. End users access the live application through the exposed service endpoint.

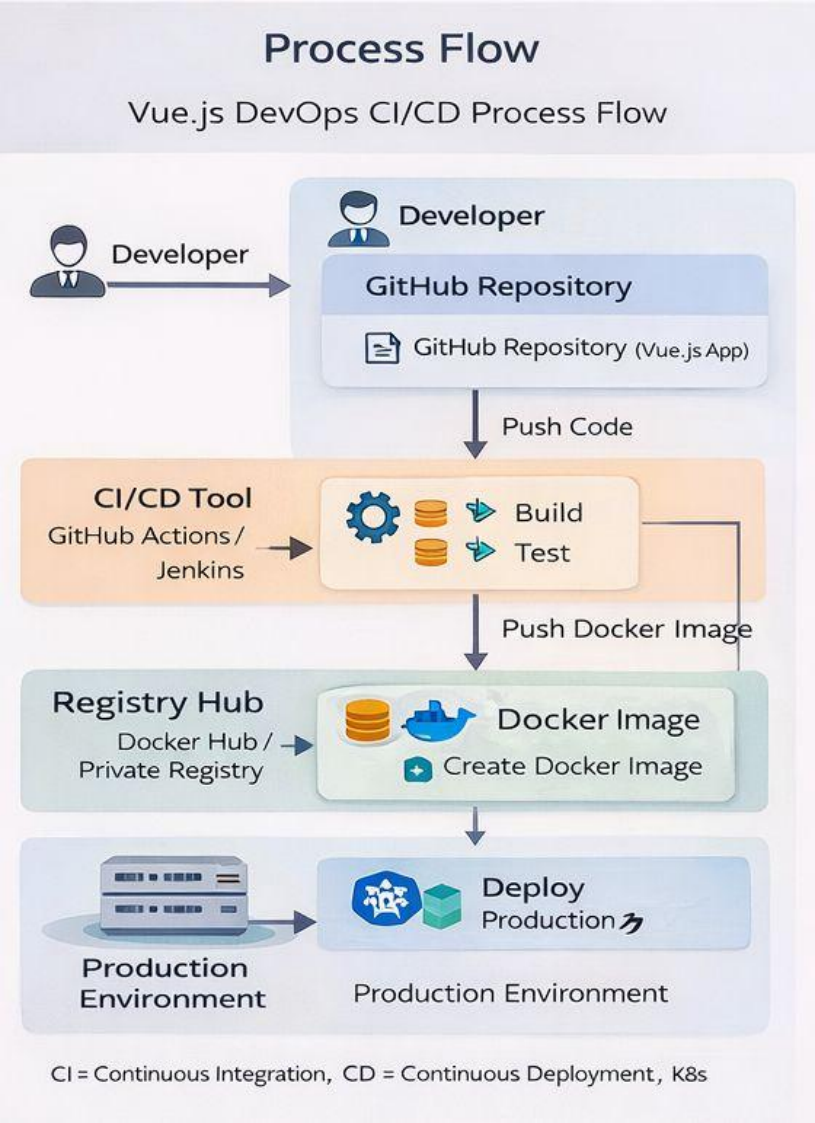


Figure 2.2.2: Detailed Process Flow Diagram

2.3 Tools & Technologies Used

Category	Tool / Technology	Purpose
Frontend	Vue.js	Used to build the user interface
Version Control	Git / GitHub	Manage and store source code
CI/CD Automation	GitHub Actions	Automates testing and deployment
Containerization	Docker	Packages the application into containers
Container Registry	Docker Hub	Stores Docker images
Orchestration	Kubernetes (K8s)	Manages and scales containers
Cloud Hosting	AWS EC2	Provides server for deployment
Build Tool	Vite / npm	Builds and bundles the app

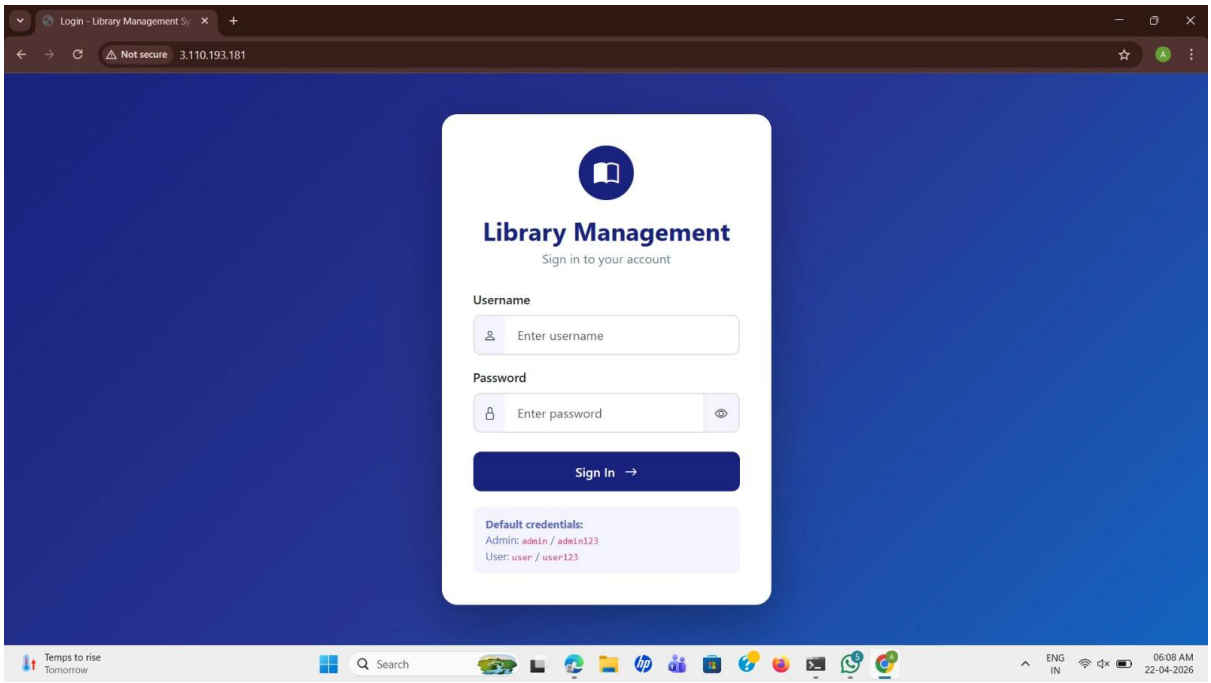
3. Result & Outputs

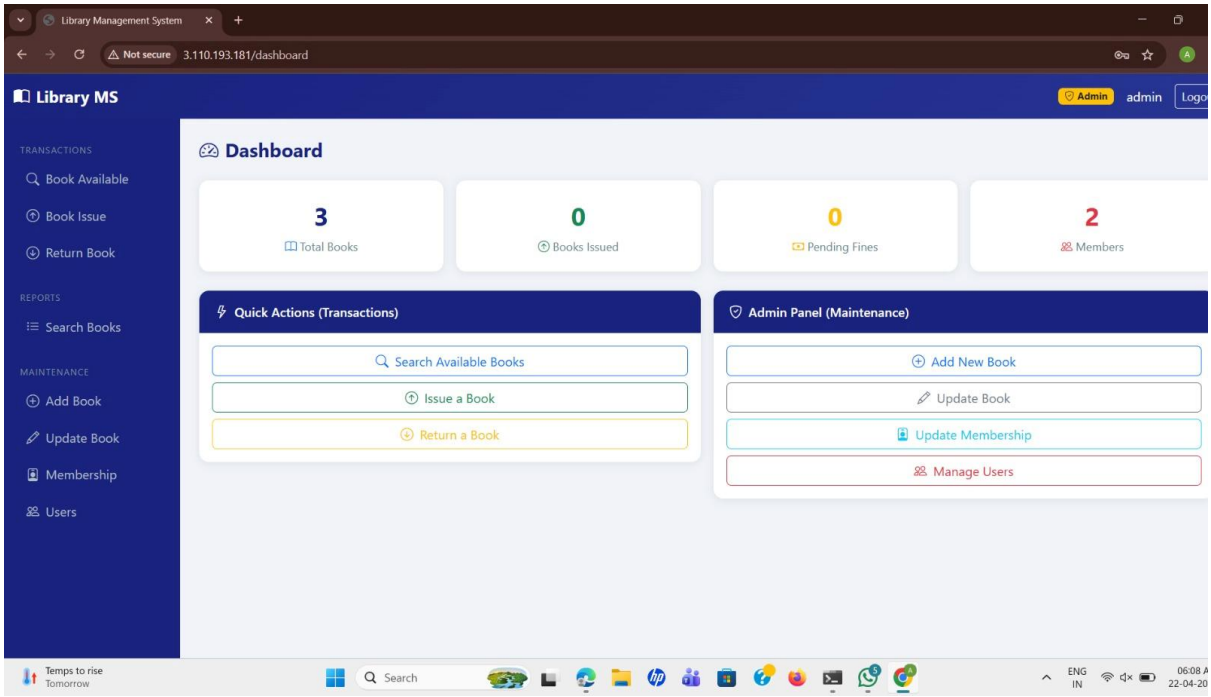
The CI/CD pipeline was successfully implemented and tested. All stages, from automated testing to deployment on Kubernetes, worked as expected. The following sections present the system interface and the key outcomes achieved.

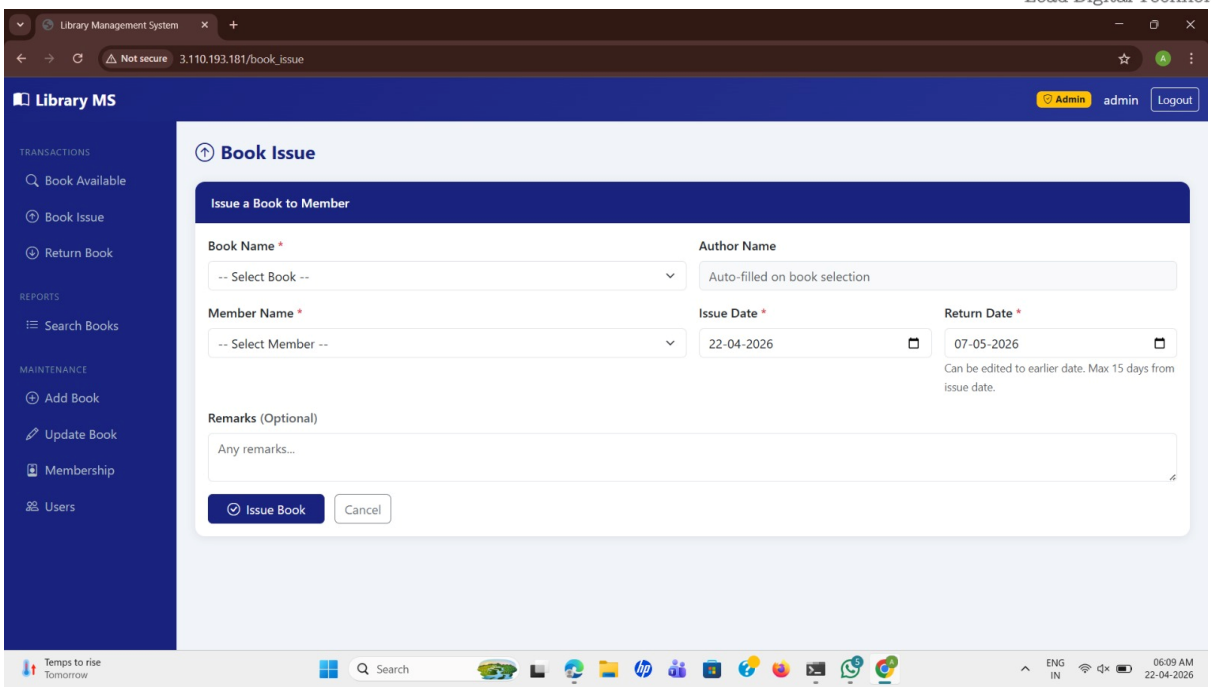
3.1 System Interface and User Experience

The system provides a seamless experience across all stages:

The user interface is responsive and loads correctly after deployment, demonstrating successful integration of all components.


Login - Library Management Sy
Not secure 3.110.193.181
Library Management
Sign in to your account
Username
Enter username
Password
Enter password
Sign In →
Default credentials:
Admin: admin / admin123
User: user / user123
Temps to rise
Tomorrow
Search
ENG
IN
06:08 AM
22-04-2026


Library Management System
Not secure 3.110.193.181/dashboard
Library MS
Admin
admin
TRANSACTIONS
Book Available
Book Issue
Return Book
REPORTS
Search Books
MAINTENANCE
Add Book
Update Book
Membership
Users
Dashboard
3
Total Books
0
Books Issued
0
Pending Fines
2
Members
Quick Actions (Transactions)
Search Available Books
Issue a Book
Return a Book
Admin Panel (Maintenance)
Add New Book
Update Book
Update Membership
Manage Users
Temps to rise
Tomorrow
Search
ENG
IN



3.2 Key Outcomes

- The automated CI/CD pipeline was successfully implemented, ensuring a smooth and continuous software delivery process.
- The system achieved faster and more reliable deployments by reducing manual intervention.
- Manual errors were significantly minimized through the use of automation at every stage.
- High scalability was achieved using Kubernetes, allowing the application to handle increased load efficiently.
- Consistent environments were maintained across development and production using Docker containers.
- Zero downtime deployment was ensured through Kubernetes rolling update strategy.

4. Conclusion

This project successfully demonstrates the implementation of a complete CI/CD pipeline for a Vue.js application using modern DevOps tools and practices. The integration of GitHub

Actions, Docker, and Kubernetes ensures automation, scalability, and reliability throughout the software delivery lifecycle.

The system reflects real-world industry practices by maintaining separation between CI and CD stages, enabling continuous testing, and deploying applications efficiently. It highlights how automation can significantly improve productivity, reduce risks, and enhance deployment speed.

Overall, the project provides a strong foundation for understanding DevOps workflows and can be extended further for enterprise-level applications.

5. Future Scope and Enhancements

The current system works well, but it can be improved further to make it more powerful and industry-ready. Some important future improvements are:

1. CI/CD Improvements
The pipeline can be improved by adding integration testing and code coverage. It can also support multiple environments for better deployment control.

2. Kubernetes Enhancements
Auto-scaling can be added to handle traffic efficiently. Ingress and Helm can improve deployment and security.

3. Monitoring & Logging
Monitoring tools can track system performance in real time. Logging systems help in debugging and issue analysis.

4. Security Enhancements
Security can be improved using vulnerability scanning and access control. Sensitive data can be stored securely.

5. Advanced Features
The system can be extended with backend services. Advanced deployment methods like GitOps can be used.